

Separation Logic and the Aliasing Issue

An Annotated Bibliography

Lani Quach (laniq@umich.edu)
University of Michigan

April 20, 2026

Overview

Introduction

Verifying imperative programs that use pointer-linked data structures on the heap is challenging, primarily because of aliasing. Aliasing occurs when multiple pointers can refer to the same location in memory, so modifying a value through one pointer may silently affect another, leading to difficulties in reasoning about program behavior.

Without tools to aid in reasoning about heap memory, many memory safety bugs may go undetected, including use-after-free errors, null pointer dereferences, and buffer overflows. In 1969, classical Hoare logic was introduced providing a framework for reasoning about imperative programs through preconditions and postconditions, but its assignment axiom assumes that variables are independent—which breaks for the case of pointers and heap allocation. To address these issues, researchers developed separation logic, which builds on top Hoare logic. This bibliography will walk through the foundations and history of separation logic and the eventual development of industry-used automated verification tools.

Summary of Key Works

In 1969, C.A.R. Hoare released a foundational paper describing the axiomatic approach to program verification [3]. In this paper, Hoare introduced the triple $\{P\} C \{Q\}$ which enabled compositional reasoning about imperative programs and became the standard for program verification.

However, Hoare’s original system only deals with simple variables and disregarded memory related functionality like heap-allocated structures or pointers. So overtime as programming languages grew to rely on dynamic memory and pointer manipulation, the more pressing this limitation became.

Eventually three decades later, the idea of separation logic was introduced by Reynolds, O’Hearn, and others, to close this gap. In 2001 a paper on the frame rule [5] was published, which states that if a portion of code only touches a certain part of memory, then all other parts of code are automatically preserved. This allows each section code to focus on verifying the small piece of the heap it actually reads and writes, making modern program verification with separation logic possible.

Reynolds’s 2002 survey paper [6] consolidated earlier work and discussed how separating conjunction, the points-to predicate, and inductive definitions for recursive data structures such as linked lists and trees all combine to form the basis of the separation logic language. The paper demonstrated these approaches by verifying several challenging algorithms such as the Schorr-Waite graph marking algorithm.

Around the same time, Ishtiaq and O’Hearn [4] independently connected separation logic to the idea of bunched implications logic (BI), creating the foundations for the assertion language. Their main contribution was showing that the separating conjunction ($*$) and the separating implication ($-*$) form a logical adjoint pair, meaning the implication is the operation that undoes the conjunction, similar to how division undoes multiplication. This relationship allows heap properties to be specified modularly: you can first describe just the portion of memory a function touches, and the frame rule can then extend that local reasoning to the full heap. They also discuss how a classical variant of the logic, rather than Reynolds’s intuitionistic version, was needed in order to reason about explicit memory deallocation.

The publishing of **Smallfoot** [1] started a shift towards more practical implementations of separation logic. Developed by researchers Berdine, Calcagno, and O’Hearn, **Smallfoot** takes programs annotated with separation logic preconditions and postconditions and performs a symbolic execution while applying proof rules along the way in order to verify memory safety properties. Although this tool required some user preparations such as annotating the program and adding loop invariants, it ultimately helped demonstrate that separation logic theories could be applied in real world tools.

As mentioned, one of the key limitations of **Smallfoot** was the need for complete specification of programs before verification could be applied. In 2009, researchers Calcagno, Distefano, O’Hearn, and Yang [2] released a

paper on bi-abduction, which automatically infers separation logic specifications procedure by procedure without the need for a whole-program analysis. This ultimately led to the creation of Facebook’s **Infer** tool, which is able to analyze millions of lines of code and catches thousands of memory safety bugs per month before those bugs reach production.

Open Research Questions

While Hoare’s logic has grown from theoretical research to full fledged industry software over time, there are still a few large challenges to be faced. Firstly, the automated tools currently developed still struggle with complex recursive data structures outside of linked lists and simple trees. Data structures such as doubly-linked lists, graphs with sharing, and arbitrary user-defined inductive structures require more expressive predicate abstractions and sophisticated automated reasoning.

Second, while separation logic for multi-threaded programs have already been developed and proven to work correctly, the tools for automated verification at the same scale as **Infer** does for sequential programs is still being actively researched. Fractional permissions and frameworks like Iris in Coq are examples of promising starts to this challenge but have yet to be developed into tools with the same level of industrial usage as **Infer**.

Finally, similarly to other areas of computer science, there is the debate in separation logic-based analyses between the tradeoffs of precision and scalability. For example, bi-abduction chooses to focus on performance over completeness—leading to the tool occasionally missing real bugs or reporting false positives. There continues to be a fundamental challenge of improving precision while maintaining the ability to analyze large codebases.

References

- [1] J. Berdine, C. Calcagno, and P. W. O’Hearn, “Smallfoot: Modular automatic assertion checking with separation logic,” in *Proc. International Symposium on Formal Methods for Components and Objects (FMCO)*, 2005.

Until this point, the majority of papers related to separation logic dealt with the theoretical foundations, and all verification in this subject required manual proof construction. Automation was needed in order to make separation logic practical for industry use. The main question was whether or not a tool could automatically check the validity of the separation logic annotations of a program, and perform the necessary symbolic reasoning about heap structures without human guidance.

In this paper, Berdine, Calcagno, and O’Hearn developed and released a tool called Smallfoot that performs forward symbolic execution of imperative programs annotated with separation logic pre- and postconditions. The tool keeps a symbolic heap state as a separation logic formula of the restricted form $\Pi \wedge \Sigma$, and applies proof rules at each step in the program in order to move this symbolic state forward. One of the main innovations in this paper was the ability to automatically infer frames from incomplete proofs in a terminating proof system for entailments $P \vdash Q$, allowing the frame rule to be applied at procedure calls without making the user manually annotate each frame. Some other innovation of this paper include symbolic execution rules for all heap-manipulating commands such as allocation and deallocation and hardwired predicates for list segments (`ls`), trees, doubly-linked lists, and xor-linked lists, with a proof theory that encodes consequences of induction beyond simple rolling/unrolling. Finally, this paper also discussing support for concurrent separation logic, including the parallel composition rule and conditional critical regions with resource invariants.

In order to validate Smallfoot, it was tested on a collection of programs including: sequential and parallel tree disposal (`disp_tree` and `par_disp_tree`), tree copying (`copy_tree`), linked list append (`append_list`, which requires reasoning at both ends of a list segment), a memory manager with infor-

mation hiding via conditional critical regions (where the free list invariant is hidden from client specifications), a pointer-transferring buffer demonstrating ownership transfer between concurrent processes (where a cell’s “permission to access” moves from sender to receiver via a resource), and concurrent xor-linked double-ended queues with parallel put/get operations outside critical sections. This paper showed that separation logic reasoning could be effectively automated for real world use.

While Smallfoot is a step in the right direction for the automation of separation logic, it still requires user-provided loop invariants, procedure pre/postconditions, and a fixed set of inductive predicates. It is unable to infer specifications automatically. It uses a strict separation model without read-sharing permissions, preventing verification of programs like readers-and-writers that require fractional permissions. Along with this, the tool does not handle general user-defined inductive predicates, and only supports a limited set of data structure descriptions like list segments.

- [2] C. Calcagno, D. Distefano, P. W. O’Hearn, and H. Yang, “Compositional shape analysis by means of bi-abduction,” *Journal of the ACM*, 2009.

Tools like Smallfoot require human input in order to create complete specifications, and when translated to large code-bases with millions of lines of code, becomes impractical. Manual annotation does not scale due to the fact that developers in industry are not going to write formal specifications. In order to create real-world impact using separation logic-based verification, these methods need to be able to analyze programs without requiring any annotations at all.

The paper introduced bi-abduction, which is a technique that automatically infers separation logic specifications. Given an assertion P describing the current heap state and a procedure call with known specification $\{A\} C \{B\}$, bi-abduction will concurrently solve for two unknowns: the anti-frame M (the additional heap needed so that $P * M$ entails A) and the frame F (the heap preserved across the call, producing postcondition $B * F$). The paper presents a proof system for abductive inference (Figure 1) with rules such as $\mapsto$ -match,

lseg-right, lseg-left, remove, and missing. These rules are used to match points-to facts, manipulate list segments, eliminate predicates that hold for the empty heap, and move unmatched predicates from the goal into the anti-frame. The BiAbd algorithm (Algorithm 2) combines abduction with frame inference by first calling Abduce to find the anti-frame M using the abduction proof system, then calling Frame to find the left-over frame F . The overall compositional analysis (Algorithm 3) works from the bottom-up, partitioning procedures into levels by their call graph, then discovering preconditions via InferPre (Algorithm 4), and finally computing postconditions via InferSpec (Algorithm 5). This all combines to produce Hoare triple summaries $\{P\}f(\vec{x})\{Q_1 \vee \dots \vee Q_k\}$ for each procedure. This is a complete analysis where each procedure is analyzed individually, its specification is inferred from its code, and the results are all combined at call sites using bi-abduction with no annotations or whole-program analysis being required.

The technique was implemented in the SpaceInvader Abductor tool and evaluated against different levels of programs. A medium-sized example such as the IEEE 1394 (firewire) device driver for Windows showed that all 121 procedures received consistent specifications, and top-level procedures like `t1394DiagPnp` were able to discover preconditions involving multiple circular linked lists with nested acyclic sublists. There were also large-scale experiments on open-source projects such as Linux kernel 2.6.25 (2.47M LOC, 101,330 procedures, 58.4% verified), Gimp 2.4.6 (708K LOC, 42.1% coverage) and OpenSSL 0.9.8g (214K LOC, 61.6% coverage). By parallelizing the operations across 8 cores, the Linux kernel analysis decreased from ~ 115 minutes to ~ 29 minutes. Eventually this technique was used to form the basis of Facebook Infer, which was deployed in production on the Facebook, Instagram, and WhatsApp codebases, analyzing diffs before code review and reporting issues before code reaches production.

Although bi-abduction forms the basis of large scale production based tools, in order to do so it trades completeness for scalability. The inferred specifications are approximations, so the analysis may result in false positives (spurious bug reports) or false negatives (missed bugs). Code pointers and concurrency are not addressed. It’s also important to note that

this technique focuses primarily on memory safety properties rather than full functional correctness. Future work in extending bi-abduction to larger program properties, improving precision, and handling concurrent programs are still being researched.

- [3] C. A. R. Hoare, “An axiomatic basis for computer programming,” *Communications of the ACM*, 1969.

Before this work, framework for formally reasoning about the correctness of imperative programs was non-existent. While programmers were able to test their code and informally reason about them, there was no way to mathematically prove that a program met its specification. Because there were no formal foundations, the reliability of software programs depended completely on human testing, which was not always correct.

Hoare’s solution introduced in this paper was an axiomatic system based on triples written as $\{P\} C \{Q\}$, where P is a precondition, C is a command, and Q is a postcondition. He provided inference rules that formed the basis of imperative programming: the assignment axiom $\{P[e/x]\} x := e \{P\}$, the rule of composition for sequential statements (D2), the rule of consequence for strengthening preconditions and weakening postconditions (D1), and the while rule with loop invariants (D3). These rules combined to form a structure similar to the program itself, by allowing proofs about programs to be constructed compositionally.

Hoare evaluated this approach through paper proofs of correctness using example programs such as a division algorithm that computes the quotient q and remainder r of x divided by y using successive subtraction, proving the postcondition $\neg y \leq r \wedge x = r + y \times q$ (Table III in the paper). The proof uses all four core rules—assignment, composition, iteration with loop invariant $x = r + y \times q$, and consequence—demonstrating that the rules are sufficient to verify properties of programs with loops. He also discussed how the axiom system could serve as a formal language definition by requiring implementations to satisfy the axioms. Eventually, the Hoare triple notation be-

came the programming standard for expressing program specifications.

The main limitation though with Hoare’s logic is the assignment axiom $P[e/x]$ assumes that program variables are independent, so assigning to x cannot affect the value of y . Pointer aliasing, where $*x = 5$ may also change $*y$ if x and y point to the same location in memory violates this assumption, rendering Hoare’s logic futile. Another issue with Hoare’s original system is it did not address heap allocation, deallocation, or recursive data structures, which were addressed in the coming decades.

- [4] S. S. Ishtiaq and P. W. O’Hearn, “BI as an assertion language for mutable data structures,” in *Proc. 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2001, pp. 14–26.

When introduced, separation logic’s assertion language and the separating conjunction did not have a clear connection to the already established logical frameworks. There was not a precise logical foundation of spatial assertions about the heap. Questions about whether the proof rules were complete, how the connectives interacted, and whether more powerful connectives (such as a spatial implication) could be introduced went unanswered due to missing foundational knowledge.

Ishtiaq and O’Hearn solved this issue by showing how separation logic assertion language is apart of the logic of bunched implications (BI), a substructural logic developed by O’Hearn and Pym. BI contains two families of connectives: the additive connectives ($\wedge$, $\rightarrow$) from standard logic, and the multiplicative connectives ($*$, $-*$) which treat assertions as resources that cannot be freely duplicated or discarded. This connection between the two showed that the separating conjunction $*$ corresponds to the multiplicative conjunction of BI, and that the heap model provides a concrete resource semantic. Importantly, the paper introduced the idea of the separating implication (“magic wand”), written as $P \multimap Q$, which states that if a disjoint heap satisfying P is added to the current heap, then Q holds when combined. The authors of this paper also showed that a Boolean (classical) version of the logic, rather than the intuitionistic version discovered by

Reynolds, was needed in order to reason about explicit memory deallocation via the `dispose` command. They provided Hoare-style proof rules for heap operations in both “Reynolds” (forward) and “backwards” forms. For example, the backwards mutation rule $\{\exists x_1, \dots, x_m. (E \mapsto E_1, E_2) * P\} E.1 := E' \{\exists x_1, \dots, x_m. (E \mapsto E', E_2) * P\}$, and showed how the magic wand can express weakest preconditions for all heap commands.

Through the use of paper proofs, the authors were able to confirm their ideas that the assertion logic and its proof rules were grounded in the BI semantics proving soundness and completeness (Theorems 4 and 5). They also demonstrated the purpose and usability of the magic wand through multiple examples including the “update as deletion followed by extension” pattern (e.g., $(x \mapsto 3, 5) * ((x \mapsto 7, 5) \multimap P)$ describes a state where updating cell x from 3 to 7 would establish P), a list disposal program with specification $\{nclist\ p\} \dots \{\mathbf{emp}\}$, and the Frame Axiom Introduction rule $\frac{\{P\}C\{Q\}}{\{P * R\}C\{Q * R\}}$ (Section 8), which they showed enables local reasoning where specifications mention only the cells in a program’s footprint. They also proved that the two conjunctions $*$ and $\wedge$ behave differently (e.g., $(x \mapsto 1, 2) * (x \mapsto 1, 2)$ is always false while $(x \mapsto 1, 2) \wedge (x \mapsto 1, 2)$ is not), showing the substructural character of the logic where contraction and weakening fail.

The paper did not address any automation but instead stuck to highly theoretical ideas. The magic wand connective was logically important for the metatheory, specifically the validity of the frame rule, but when placed in practice was not as simple to implement. Therefore due to the complexity of reasoning with the magic wand, automated tools like Smallfoot and Infer generally tend to avoid it. Along with this, the BI framework introduced in this paper handles only first-order assertions, meaning it does not support quantification over predicates or assertions themselves, so extending the same logic to higher-order separation logic and concurrent settings would require further work by researchers.

- [5] P. W. O’Hearn, J. C. Reynolds, and H. Yang, “Local reasoning about programs that alter data structures,” in *Proc. Computer Science Logic (CSL)*, 2001.

When it comes to verifying programs in the context of a larger system, it requires specifying the entire heap state in order to help reason about whether a program behaves correctly without any unintended interference from other parts of memory. For example, if a function is only supposed to manipulate a linked list, in its specification it must still account for all other data structures in the program. This makes modular reasoning, or verifying each component individually and composing the results, virtually infeasible.

This paper introduces the frame rule, which describes the principle of local reasoning. The frame rule states that if $\{P\} C \{Q\}$ holds and R describes heap disjoint from what C accesses, then $\{P * R\} C \{Q * R\}$, provided that C does not modify any free variables of R . This extends classical Hoare logic by what the authors call “small axioms”, specific parts of a program are specified and verified using only the portion of the heap it actually touches rather than attempting to verify the memory as a whole. For example, the mutation axiom is $\{E \mapsto -\} [E] := F \{E \mapsto F\}$, which specifies only the single cell at address E . This also guarantees that any additional heap described by R will be preserved, even without any explicit reasoning.

The authors evaluated the frame rule by providing paper proofs of its solidity by demonstrating the modular verification of tree copying (**CopyTree**), which requires the frame rule to establish that recursive calls on subtrees do not interfere with each other, and difference-linked list operations (**enqueue**, **dequeue**, **setrptr**, **setlptr**) that demonstrate address arithmetic on a compact doubly-linked list representation storing $n - p$ instead of separate next and previous pointers. They proved that compared to global specifications, specifications using the frame rule are ultimately more concise, and that after verifying separate components they can be composed. For instance, the **CopyTree** spec $\{tree \tau p\} \text{CopyTree}(p; q) \{tree \tau p * tree \tau q\}$ mentions only the relevant tree, not the entire heap. The frame rule is shown to be complete in the sense that any semantic consequence of a valid specification $\{P\} C \{Q\}$ can be derived using the frame rule together with consequence, auxiliary variable elimination, and substitution.

Although this paper shows how extensions of Hoare logic

can be beneficial, the overall approach to the frame rule and verification still remains manual. Having to identify the appropriate frame for a given piece of code requires human insight along with the assumption of a distinct separation between the footprint and the frame, which can be difficult to establish for certain programs with complex memory systems. The paper identified these issues and touched on the possibility of automating frame inference, which was later addressed by bi-abduction.

- [6] J. C. Reynolds, “Separation logic: A logic for shared mutable data structures,” in *Proc. IEEE Symposium on Logic in Computer Science (LICS)*, 2002.

Previous research had verification via Hoare logic on programs that manipulate heap-allocated, pointer-linked data structures cannot be done properly. The difficulty lies in the principle of aliasing, which is when multiple pointers can refer to the same memory location. In order to reason about the effect of heap mutations, it would require considering all possible aliasing relationships, which is beyond the abilities of proof theory. For example, even simple operations on linked lists would require timely case analysis about which pointers might overlap within memory.

In order to solve this issue of aliasing, Reynolds published this paper on separation logic, extending Hoare logic with spatial connectives for describing heap structure. More specifically, the separating conjunction $P * Q$, which asserts that P and Q hold on disjoint parts of the heap, directly addressing the aliasing problem. The points-to predicate $x \mapsto v$ asserts that location x holds value v and nothing else, and **emp** asserts the empty heap. He introduced the separating implication (magic wand) $P \multimap Q$ and provided inductive predicate definitions for data structures such as list segments ($list\ \alpha\ (i, j)$), doubly-linked list segments ($dlist$), xor-linked lists ($xlist$), trees, and dags. He presented local, global, and backwards-reasoning forms of proof rules for mutation, lookup, allocation, and deallocation, as well as the frame rule and rules for recursive procedures. He also extended the programming lan-

guage to support dynamically allocated arrays with an iterated separating conjunction $\odot_{i=m}^n p(i)$.

Through a series of paper proofs of programs with variable levels of verification difficulty. These proofs included in-place linked list reversal (where the invariant simplifies from Equation (2) to $\exists \alpha, \beta. \text{list } \alpha \ i * \text{list } \beta \ j * \text{list } \gamma \ x \wedge \alpha_0^\dagger = \alpha^\dagger \cdot \beta$), list element deletion, tree copying for both tree and dag inputs (where the dag case required assertion variables to express that all heap properties are preserved), a cyclic buffer using iterated separating conjunction, a subsequence-listing program that precisely characterizes sharing (proving the output uses $(2 \times 2^{|\alpha|}) - 1$ cons-pairs), and the Schorr-Waite graph marking algorithm proved by Yang. The Schorr-Waite verification had previously defeated verification approaches due to its use of pointer reversal during traversal, which creates complex temporary aliasing patterns that are hard to verify.

Similarly to O’Hearn and the original Hoare papers, the proof rules in this paper are designed for manual verification, meaning no automated tools were created for this research subject. The logic was only ever tested and restricted to sequential programs, meaning programs that introduced concurrent shared-memory were not addressed in this paper. Reynolds noted this limitation and described the need for extensions to handle cases such as unrestricted address arithmetic, dynamically allocated arrays, and recursive procedures, some of which were sketched out in the paper.

Lani Quach
CSE 590 - Advanced Programming Languages
April 17th , 2026

Separation Logic and the Aliasing Issue

Team: Lani Quach

Papers:

1. Hoare 1969 ("An Axiomatic Basis for Computer Programming")
 - The foundational paper introducing Hoare triples $\{P\}C\{Q\}$.
2. O'Hearn, Reynolds, Yang 2001 ("Local Reasoning about Programs that Alter Data Structures")
 - Introduces the frame rule, which describes local reasoning about code that only touches part of the heap.
3. Ishtiaq, O'Hearn 2001 ("BI as an Assertion Language for Mutable Data Structures")
 - Connects separation logic's assertion language to the logic of bunched implications (BI).
4. Reynolds 2002 ("Separation Logic: A Logic for Shared Mutable Data Structures")
 - The original paper introduces the separating conjunction and core proof rules for reasoning about heap-manipulation programs.
5. Berdine, Calcagno, O'Hearn 2005 ("Smallfoot: Modular Automatic Assertion Checking with Separation Logic")
 - Describes Smallfoot, the first automated verifier for separation logic.
6. Calcagno, Distefano, O'Hearn, Yang 2009 ("Compositional Shape Analysis by Means of Bi-Abduction")
 - The technique behind Facebook Infer, bi-abduction.

Topic Description: Separation logic builds off of Hoare logic, which reasons about programs that deal with manipulating pointers and heap-allocated memory. The main challenge that gets described with separation logic is the idea of aliasing (when multiple pointers can refer to the same memory location). So by modifying one pointer, you may cause silent effects on the other, making verification extremely difficult with standard Hoare logic. Separation logic solves this problem by introducing the separating conjunction which states that two pieces of heap memory are disjoint. Separation logic also introduces the frame rule which allows reasoning locally about only the memory a piece of code actually touches. Real world examples of this are shown in verification tools such as Facebook's Infer, which uses separation logic and bi-abduction to find memory safety bugs in its product's codebases.

Implementation Plan: I'm planning to build a small separation logic verifier inspired by Smallfoot. The tool will take simple imperative programs with heap operations (allocation, deallocation, pointer read, pointer write) annotated with pre and postconditions written in separation logic, and check whether those annotations hold by stepping through the program. The assertion language will include the core separation logic primitives: the empty heap predicate (emp), the points-to predicate ($x \mapsto v$), and the separating conjunction ($*$). Internally, I'll represent the heap as a set of disjoint points-to facts and implement a few of the key proof rules, most importantly the frame rule, which is what makes separation logic useful in the first place. My goal is to verify a couple of small, concrete examples, such as swapping the values of two heap cells or a short straight-line program that allocates, writes to, and frees memory. If that goes well, I'd

like to extend it to handle a simple linked list operation, but I'm treating that as a stretch goal rather than a core deliverable. This plan may change as I implement more of the backend.